CSE 152A Fall 2022 – Assignment 1

- Assignment Published On: **Wed, Oct 05, 2022**
- Due On: **Fri, Oct 14, 2022 11:59 PM (Pacific Time)**

Instructions

Please answer the questions below using Python in the attached Jupyter notebook and follow the guidelines below.

- The assignment is to be completed **individually**. For more details, please follow the Assignment Policy and Collaboration Policy posted on lecture slides.
- All the solutions must be written in your Jupyter notebook.
- While filling the cells in the notebook, please make the datatype as a PDF and submit both the notebook and the PDF (i.e. File → Download as → PDF via LaTeX (.pdf) in the Jupyter Notebook).
- You may use basic libraries (e.g. `NumPy`, `SciPy`, etc) but you are not allowed to use open source code that directly solves the problems. Feel free to ask the instructor and the teaching assistants if you are unsure about the allowed libraries.
- It is highly recommended that you begin early and work on the assignment.

Late Policy: Assignments submitted late will receive a 10% grade reduction for each 12 hours late (thus a 30% reduction for one day late).

Problem 1: Photometric Stereo [20 pts]

The goal of this problem is to implement a photometric stereo system.

Data files attached to problem and test cases are provided in the dataset zip file.

Hint: Jupyter stores data files in a folder called "Data" by default.

Data

In the zip file, you are provided images as follows. These images are stored in `.pickle` files which were previously provided by Satya Mallick. Each `.pickle` file contains

- `sphere_images.pickle`
- `CO_152_152_152_.pickle`
- `CO_152_152_152_Left.pickle`

You will find that the objects the object in `sphere_images.pickle` are of the following form:

```
# Sphere
# We have 7 images for each pixel and assume data type is uint8
import pickle
import numpy as np
import matplotlib.pyplot as plt
pickle_in = open("Data/sphere_images.pickle","rb")
sphere_images = pickle.load(pickle_in,encoding='bytes')
# plot 1st, 2nd, 3rd, 4th, 5th, 6th, 7th image of the sphere
print("Shape: ", sphere_images.shape)
for i in range(7):
    plt.imshow(sphere_images[:,:,i],cmap='gray')
    plt.show()
print("Shape: ", sphere_images[:,:,0].shape)
print(sphere_images[:,:,0])
```

```
Shape:  (152, 152, 7)
```

```
Shape:  (152, 152)
[[ 0 0 0 ... 0 0 0]
 [ 0 0 0 ... 0 0 0]
 [ 0 0 0 ... 0 0 0]
 ...
```

Each image provides lighting information about the object in the scene. Your program should have two parts:

- Read in the images and corresponding light source directions, and estimate the surface normals and albedo map.
- Using the estimated surface normals and albedo map, implement the Lambertian model to render an image of the object from the same viewpoint as the "sun_images" function. Note that you will typically need to run the "sun_images" function before you can run the "sphere_images" function, meaning it will take a while.

```
import time, os
from mpl_toolkits.mpl_toolkit
import pickle
def sun_images(path):
    '''This function requires the function to be in the parent
    directory and the data folder to be in the current
    directory. It takes in the path to the data folder
    (relative to the current directory)
    and returns a numpy array of the 152x152x152x3 images.
    '''
    ts = time.strftime("%Y-%m-%d-%H-%M-%S")
    print("Start time: ", ts)
    # 152 x 152 x 152 x 3 array of images
    images = np.zeros((152,152,152,3))
    # 152 x 152 x 152 x 3 array of light sources
    lights = np.zeros((152,152,152,3))
    # loop through the files in the data folder
    for i in range(1,153):
        if i % 10 == 0:
            print("Loading image: ", i)
        if os.path.exists(path+"/CO_152_152_152_.pickle"):
            pickle_in = open(path+"/CO_152_152_152_.pickle","rb")
            images[:,:,:,i-1] = pickle.load(pickle_in,encoding='bytes')
        if os.path.exists(path+"/CO_152_152_152_Left.pickle"):
            pickle_in = open(path+"/CO_152_152_152_Left.pickle","rb")
            lights[:,:,:,i-1] = pickle.load(pickle_in,encoding='bytes')
    print("End time: ", ts)
    return images, lights
images, lights = sun_images("./Data")
print("Shape: ", images.shape)
print("Shape: ", lights.shape)
```

```
Start time:  2022-10-05-00-00-00
Loading image:  10
Loading image:  20
Loading image:  30
Loading image:  40
Loading image:  50
Loading image:  60
Loading image:  70
Loading image:  80
Loading image:  90
Loading image:  100
Loading image:  110
Loading image:  120
Loading image:  130
Loading image:  140
Loading image:  150
End time:  2022-10-05-00-00-00
Shape:  (152, 152, 152, 3)
Shape:  (152, 152, 152, 3)
```

1(a) Photometric Stereo [6 pts]

- Read in the images and corresponding light source directions, and estimate the surface normals and albedo map.
- Display the estimated surface normals and albedo map as a grayscale image of the form [height,width,channel].

```
# Your code here
# Estimate albedo and surface normals
def estimate_albedo_normals(images, lights):
    '''This function takes in the images and lights
    and returns the albedo and surface normals.
    It assumes the images and lights are in the form
    152x152x152x3.
    '''
    # 152 x 152 x 3 array of albedo
    albedo = np.zeros((152,152,3))
    # 152 x 152 x 3 array of surface normals
    normals = np.zeros((152,152,3))
    # loop through the pixels
    for i in range(152):
        for j in range(152):
            # 7 x 3 matrix of light sources
            L = lights[i,j,:,:]
            # 7 x 1 vector of intensities
            I = images[i,j,:,0]
            # 3 x 1 vector of albedo * normal
            b = np.linalg.inv(L.T@L)@L.T@I
            # albedo is the norm of b
            albedo[i,j] = np.linalg.norm(b)
            # normals is b / albedo
            if albedo[i,j] > 0:
                normals[i,j] = b / albedo[i,j]
    return albedo, normals
albedo, normals = f(images, lights)
```

1(b) Display outputs and save. (.tif and .npy files in .zip file)

- The estimated albedo map.
- The estimated surface normals by showing both
  - Needle-map
  - Three image showing components of surface normal.
- Save the surface normals as a .npy file.

```
# Your code here
# Display albedo
print("Shape: ", albedo.shape)
plt.imshow(albedo, cmap='gray')
plt.show()
# Display normals as 3 images
print("Shape: ", normals.shape)
plt.imshow(normals[:,:,0], cmap='gray')
plt.show()
plt.imshow(normals[:,:,1], cmap='gray')
plt.show()
plt.imshow(normals[:,:,2], cmap='gray')
plt.show()
# Save normals as .npy file
np.save("./Data/normals.npy", normals)
```

```
Shape:  (152, 152, 3)
```

```
Shape:  (152, 152, 3)
```

1(c) Display outputs using .tif files

- Albedo Image
- Normals as three separate channels

Albedo Image

Normals as three separate channels

Needle-map

Wireframe of depth map

```
# Your code here
# Estimate the height map using the method from part 1 part (e)(i) and (ii)
# of the photo stereopsis PAMI paper. You may write your own method or choose the
# method that gives you the best results. Display the height map as a surface plot.
# Save the height map as a .npy file.
def estimate(normals, normals_term,depth):
    '''This function takes in the normals and returns the surface in the different images
    '''
    # initialize the height map
    height = np.zeros(152,152)
    # 152 x 152 x 2 array of gradients
    gradients = np.zeros(152,152,2)
    # loop through the pixels
    for i in range(152):
        for j in range(152):
            # 2 x 2 matrix of normals
            N = normals[i,j,:2]
            # 2 x 1 vector of gradients
            g = gradients[i,j,:]
            # 2 x 2 matrix of normals * normals.T
            NN = N.T@N
            # 2 x 2 matrix of normals * normals.T - I
            NN_I = NN - np.eye(2)
            # 2 x 2 matrix of normals * normals.T - I * gradients
            NN_I_g = NN_I@g
            # 2 x 1 vector of normals * normals_term
            N_term = N@normals_term[i,j]
            # 2 x 1 vector of NN_I_g - N_term
            diff = NN_I_g - N_term
            # 2 x 2 matrix of NN_I
            NN_I = NN_I
            # 2 x 1 vector of diff
            b = diff
            # 2 x 2 matrix of NN_I.T * NN_I
            A = NN_I.T@NN_I
            # 2 x 1 vector of NN_I.T * b
            c = NN_I.T@b
            # 2 x 1 vector of gradients
            gradients[i,j,:] = np.linalg.inv(A)@c
    # loop through the pixels
    for i in range(152):
        for j in range(152):
            # 1 x 1 vector of height
            h = height[i,j]
            # 2 x 1 vector of gradients
            g = gradients[i,j,:]
            # 1 x 1 vector of height + gradients[0] + gradients[1]
            height[i,j] = h + g[0] + g[1]
    return height
height = f(normals, normals_term, depth)
```

```
# Your code here
# Display height map as a surface plot
import mpl_toolkits.mplot3d as plt3d
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
# Load the height map
height = np.load("./Data/height.npy")
# Create a figure
fig = plt.figure(figure=(10,10))
# Create a 3D axes
ax = fig.add_subplot(1,1,1,projection='3d')
# Create a meshgrid
x, y = np.meshgrid(range(height.shape[0]), range(height.shape[1]))
# Plot the surface
ax.plot_surface(x, y, height, cmap='gray')
# Show the plot
plt.show()
```


1(d) Reflectance surrounds [5 pts]

Calculate the surface normals and display them as a quiver plot using quiver in matplotlib pyplot or Plotly. Assume that the surface normals are given by

$$-\frac{\partial z}{\partial x}, -\frac{\partial z}{\partial y}, 1$$

(1)

Also, recall, that each normal vector should be normalized to unit length.

```
# Compute and plot the surface normals
# Load the height map and compute the gradients in x and y directions
import numpy
import matplotlib
import matplotlib.pyplot as plt
import mpl_toolkits.mplot3d as plt3d
# Load the height map
height = np.load("./Data/height.npy")
# Create a figure and a 3D axes
fig = plt.figure(figure=(10,10))
ax = fig.add_subplot(1,1,1,projection='3d')
# Create a meshgrid
x, y = np.meshgrid(range(height.shape[0]), range(height.shape[1]))
# Compute the gradients in x and y directions
dx, dy = np.gradient(height)
# Compute the surface normals
nx, ny, nz = -dx, -dy, 1
# Normalize the surface normals
norm = np.sqrt(nx**2 + ny**2 + nz**2)
nx, ny, nz = nx/norm, ny/norm, nz/norm
# Plot the surface
ax.plot_surface(x, y, height, cmap='gray')
# Plot the quiver
ax.quiver(x, y, height, nx, ny, nz, length=0.5, normalize=True)
# Show the plot
plt.show()
```


Problem 2: Image Rendering [20 points]

In this exercise, we will render an image using the Phong reflectance model with ambient lighting and a perfect diffuse reflectance model. We will use two albedo maps, one uniform and one that is cross-eyed. The face height map, the light sources, and the two albedo maps are provided in the dataset zip file. Further, each row of the `lightsources` sample contains unit length directions. The face height map has been flattened and scaled to a mesh. The cross-eyed albedo map is provided in a Numpy compressed format.

Hint: Please make good use of unit vectors and broadcasting when implementing the equations.

2(a) Plot the face in 3D [5 pts]

Plot the face using the provided `height_values` and `uv_grid` arrays.

```
import numpy as np
import matplotlib.pyplot as plt
import mpl_toolkits.mplot3d as plt3d
height = np.load("./Data/height.npy")
uv_grid = np.load("./Data/uv_grid.npy")
# Plot the surface using the height_values and uv_grid
# Create a figure and a 3D axes
fig = plt.figure(figure=(10,10))
ax = fig.add_subplot(1,1,1,projection='3d')
# Create a meshgrid
x, y = np.meshgrid(range(height.shape[0]), range(height.shape[1]))
# Plot the surface
ax.plot_surface(x, y, height, cmap='gray')
# Show the plot
plt.show()
```

2(b) Plot the face in R (5 pts)

Using both the heightmap and the albedo, plot the face using plot_surface. Do this for both albedos. Explain what you see.

```
# Plot the face in R using both albedo maps using matplotlib
import numpy as np
import matplotlib.pyplot as plt
height = np.load("./Data/height.npy")
uv_grid = np.load("./Data/uv_grid.npy")
# Create a figure and a 3D axes
fig = plt.figure(figure=(10,10))
ax = fig.add_subplot(1,1,1,projection='3d')
# Create a meshgrid
x, y = np.meshgrid(range(height.shape[0]), range(height.shape[1]))
# Plot the surface
ax.plot_surface(x, y, height, cmap='gray')
# Show the plot
plt.show
```

2(c) Plot the face in R (5 pts)

Using both the heightmap and the albedo, plot the face using plot_surface. Do this for both albedos. Explain what you see.

```
import numpy as np
import matplotlib.pyplot as plt
import mpl_toolkits.mplot3d as plt3d
height = np.load("./Data/height.npy")
uv_grid = np.load("./Data/uv_grid.npy")
# Create a figure and a 3D axes
fig = plt.figure(figure=(10,10))
ax = fig.add_subplot(1,1,1,projection='3d')
# Create a meshgrid
x, y = np.meshgrid(range(height.shape[0]), range(height.shape[1]))
# Plot the surface
ax.plot_surface(x, y, height, cmap='gray')
# Show the plot
plt.show
```

2(d) Render normals [5 pts]

Calculate the surface normals and display them as a quiver plot using quiver in matplotlib pyplot or Plotly. Assume that the surface normals are given by

$$-\frac{\partial z}{\partial x}, -\frac{\partial z}{\partial y}, 1$$

(2)

Also, recall, that each normal vector should be normalized to unit length.

```
# Compute and plot the surface normals
# Load the height map and compute the gradients in x and y directions
import numpy
import matplotlib
import matplotlib.pyplot as plt
import mpl_toolkits.mplot3d as plt3d
# Load the height map
height = np.load("./Data/height.npy")
# Create a figure and a 3D axes
fig = plt.figure(figure=(10,10))
ax = fig.add_subplot(1,1,1,projection='3d')
# Create a meshgrid
x, y = np.meshgrid(range(height.shape[0]), range(height.shape[1]))
# Compute the gradients in x and y directions
dx, dy = np.gradient(height)
# Compute the surface normals
nx, ny, nz = -dx, -dy, 1
# Normalize the surface normals
norm = np.sqrt(nx**2 + ny**2 + nz**2)
nx, ny, nz = nx/norm, ny/norm, nz/norm
# Plot the surface
ax.plot_surface(x, y, height, cmap='gray')
# Plot the quiver
ax.quiver(x, y, height, nx, ny, nz, length=0.5, normalize=True)
# Show the plot
plt.show()
```


Problem 3: Homogeneous Coordinates and Vanishing Points [20 points]

In this problem, we will be using the following notation: $\tilde{x}$ denotes the homogeneous coordinates of a point x, $\tilde{x} = [x, y, 1]^T$ denotes the 2D pixel coordinates in homogeneous coordinates, $\tilde{x} = [x, y, z]^T$ denotes a point in 3D in homogeneous coordinates, $\tilde{x} = [x, y, z, 1]^T$ denotes the same 3D point. Dividing by the third coordinate of a homogeneous point $\tilde{x}$, $\frac{\tilde{x}}{z}$ converts it back to its 2D equivalent.

Consider a line in the 2D scene, whose equation is given by $ax + by + c = 0$. We can equivalently be written as $\tilde{l} \cdot \tilde{x} = 0$ where $\tilde{l} = [a, b, c]^T$ and $\tilde{x} = [x, y, 1]^T$. Noting that x is a homogeneous representation of x, we can write the equation of the line in homogeneous coordinates as $\tilde{l}^T \tilde{x} = 0$. Similarly, the equation of a plane in 3D can be written as $\tilde{p} \cdot \tilde{x} = 0$ where $\tilde{p} = [a, b, c, d]^T$ and $\tilde{x} = [x, y, z, 1]^T$. So the plane represented by $\tilde{p}$ intersects the 3D point $\tilde{x}$ if $\tilde{p}^T \tilde{x} = 0$, or $ax + by + cz + d = 0$.

Although $\tilde{x}$ is defined as the point $[x, y, z, 1]^T$, it represents the same point in 3D as any other point in homogeneous coordinates $\alpha \tilde{x}$. We can therefore look for a point that satisfies $\tilde{p}^T \tilde{x} = 0$ and $\tilde{p}^T \tilde{x} = 0$ at the same time, as long as $\tilde{x}$ satisfies both equations. Stacking the two equations together, we get $\begin{bmatrix} \tilde{p}^T \\ \tilde{p}^T \end{bmatrix} \tilde{x} = 0$. Solving this system of equations will give us the intersection of the two planes represented by $\tilde{p}$ and $\tilde{p}$.

Read the points and lines in homogeneous coordinates from the homogeneous.npz file and solve $\tilde{x}$.

Hint: Use `numpy.linalg.solve`.

Note: Do not forget to convert the homogeneous representations for the points $\tilde{x}, \tilde{x}$ and $\tilde{x}$ back to 2D points, you can write $x = \tilde{x}$ and so on and so on. (1 Mark)

```
# Your code here
import numpy as np
# Load the points and lines from the file
points = np.load("./Data/homogeneous.npz")['points']
lines = np.load("./Data/homogeneous.npz")['lines']
# Print the points and lines
print("Points: ", points)
print("Lines: ", lines)
# Solve for the intersection of the first two points and the first line
intersection = np.linalg.solve(points[:2], lines[0])
# Convert the intersection to 2D
intersection = intersection[:2] / intersection[2]
# Print the intersection
print("Intersection: ", intersection)
```

```
Points:  [[ 1.  2.  1.]
 [ 3.  4.  1.]
 [ 5.  6.  1.]
 [ 7.  8.  1.]
 [ 9. 10.  1.]]
Lines:  [[ 1.  1. -3.]
 [ 1. -1.  1.]
 [ 2.  1. -4.]
 [ 1.  2. -5.]
 [ 3.  1. -6.]]
Intersection:  [1. 2.]
```

Write a function to compute the intersection of two lines and a function to compute the line through two homogeneous points, e.g., `intersection` of parallel and `line_through_points` for the 3D points of each plane.

```
# Your code here
import numpy as np
# Load the points and lines from the file
points = np.load("./Data/homogeneous.npz")['points']
lines = np.load("./Data/homogeneous.npz")['lines']
# Print the points and lines
print("Points: ", points)
print("Lines: ", lines)
# Solve for the intersection of the first two points and the first line
intersection = np.linalg.solve(points[:2], lines[0])
# Convert the intersection to 2D
intersection = intersection[:2] / intersection[2]
# Print the intersection
print("Intersection: ", intersection)
# Solve for the intersection of the second two points and the second line
intersection = np.linalg.solve(points[2:4], lines[1])
# Convert the intersection to 2D
intersection = intersection[:2] / intersection[2]
# Print the intersection
print("Intersection: ", intersection)
# Solve for the intersection of the third two points and the third line
intersection = np.linalg.solve(points[4:6], lines[2])
# Convert the intersection to 2D
intersection = intersection[:2] / intersection[2]
# Print the intersection
print("Intersection: ", intersection)
# Solve for the intersection of the fourth two points and the fourth line
intersection = np.linalg.solve(points[6:8], lines[3])
# Convert the intersection to 2D
intersection = intersection[:2] / intersection[2]
# Print the intersection
print("Intersection: ", intersection)
# Solve for the intersection of the fifth two points and the fifth line
intersection = np.linalg.solve(points[8:10], lines[4])
# Convert the intersection to 2D
intersection = intersection[:2] / intersection[2]
# Print the intersection
print("Intersection: ", intersection)
```

```
Points:  [[ 1.  2.  1.]
 [ 3.  4.  1.]
 [ 5.  6.  1.]
 [ 7.  8.  1.]
 [ 9. 10.  1.]]
Lines:  [[ 1.  1. -3.]
 [ 1. -1.  1.]
 [ 2.  1. -4.]
 [ 1.  2. -5.]
 [ 3.  1. -6.]]
Intersection:  [1. 2.]
Intersection:  [1. 2.]
Intersection:  [1. 2.]
Intersection:  [1. 2.]
Intersection:  [1. 2.]
```

Problem 4: Camera Matrices and Rigid-Body Transformations [20 points]

Consider a world coordinate system W oriented at the origin O_W with axes given by unit vectors $\hat{i} = [1, 0, 0]^T$, $\hat{j} = [0, 1, 0]^T$ and $\hat{k} = [0, 0, 1]^T$. We can a column where footballs stand for a camera and a bar where footballs stand for a point.

- **Step1** Consider another coordinate system, with unit vectors along axis of the camera axes as $\hat{i} = [0.5, 0.4, 0.7]^T$ and $\hat{j} = [-0.4305, 0.8907, 0.0990]^T$. The line unit vector $\hat{k}$ along the line which is orthogonal to both $\hat{i}$ and $\hat{j}$. Is there a unique such unit vector? If not, choose the one that makes a right-handed system.
- **Step2** Translate the camera coordinate system by $[0, 0, 1]^T$.
- **Step3** Rotate the camera coordinate system about the $\hat{k}$ axis by $\frac{\pi}{2}$ in a counterclockwise direction.

```
# Step1 Find the unit vector along the line
import numpy as np
i = np.array([0.5, 0.4, 0.7])
j = np.array([-0.4305, 0.8907, 0.0990])
# Find the unit vector along the line
k = np.cross(i, j)
# Print the unit vector along the line
print("Unit vector along the line: ", k)
# Step2 Translate the camera coordinate system by [0, 0, 1]
t = np.array([0, 0, 1])
# Print the translation vector
print("Translation vector: ", t)
# Step3 Rotate the camera coordinate system about the k axis by pi/2 in a counterclockwise direction
theta = np.pi/2
# Rotation matrix
R = np.array([[np.cos(theta), -np.sin(theta), 0], [np.sin(theta), np.cos(theta), 0], [0, 0, 1]])
# Print the rotation matrix
print("Rotation matrix: ", R)
```

```
Unit vector along the line:  [ 0.7  -0.1  -0.4]
Translation vector:  [0 0 1]
Rotation matrix:  [[ 0 -1  0]
 [ 1  0  0]
 [ 0  0  1]]
```

- **Step4** Write the location of the line passing through the points $[1, 1, 1]^T$ and $[1, 2, 1]^T$ in the camera coordinates(This term refer does it a vector that is a homogeneous representation for the line.

(1 Mark)

```
# Step4 Write the location of the line passing through the points [1,1,1]^T and [1,2,1]^T in the camera coordinates
import numpy as np
# Step1 Find the unit vector along the line
i = np.array([0.5, 0.4, 0.7])
j = np.array([-0.4305, 0.8907, 0.0990])
# Find the unit vector along the line
k = np.cross(i, j)
# Step2 Translate the camera coordinate system by [0, 0, 1]
t = np.array([0, 0, 1])
# Step3 Rotate the camera coordinate system about the k axis by pi/2 in a counterclockwise direction
theta = np.pi/2
# Rotation matrix
R = np.array([[np.cos(theta), -np.sin(theta), 0], [np.sin(theta), np.cos(theta), 0], [0, 0, 1]])
# Step4 Write the location of the line passing through the points [1,1,1]^T and [1,2,1]^T in the camera coordinates
# Line through the points
line = np.cross(np.array([1, 1, 1]), np.array([1, 2, 1]))
# Print the line
print("Line: ", line)
```

```
Line:  [ 1 -1  1]
```

- **Step5** Write the location of the line in $\hat{i} - \hat{j} + \hat{k}$ and $\hat{i} + 2\hat{j} + \hat{k}$. What is the epipolar relationship between these two lines in the Euclidean plane? What is their intersection (hint: What is their intersection in Homogeneous Coordinates?

(1 Mark)

```
# Step5 Write the location of the line in i - j + k and i + 2j + k. What is the epipolar relationship between these two lines in the Euclidean plane? What is their intersection (hint: What is their intersection in Homogeneous Coordinates?
import numpy as np
# Step1 Find the unit vector along the line
i = np.array([0.5, 0.4, 0.7])
j = np.array([-0.4305, 0.8907, 0.0990])
# Find the unit vector along the line
k = np.cross(i, j)
# Step2 Translate the camera coordinate system by [0, 0, 1]
t = np.array([0, 0, 1])
# Step3 Rotate the camera coordinate system about the k axis by pi/2 in a counterclockwise direction
theta = np.pi/2
# Rotation matrix
R = np.array([[np.cos(theta), -np.sin(theta), 0], [np.sin(theta), np.cos(theta), 0], [0, 0, 1]])
# Step4 Write the location of the line passing through the points [1,1,1]^T and [1,2,1]^T in the camera coordinates
# Line through the points
line = np.cross(np.array([1, 1, 1]), np.array([1, 2, 1]))
# Print the line
print("Line: ", line)
# Step5 Write the location of the line in i - j + k and i + 2j + k. What is the epipolar relationship between these two lines in the Euclidean plane? What is their intersection (hint: What is their intersection in Homogeneous Coordinates?
# Line through the points
line = np.cross(np.array([1, -1, 1]), np.array([1, 2, 1]))
# Print the line
print("Line: ", line)
```

```
Line:  [ 1 -1  1]
Line:  [-3  0  3]
```

- **Step6** Find the relative pose between the two cameras, i.e. find a rotation matrix R and a translation vector t such that $\tilde{x}_2 = R\tilde{x}_1 + t$.

(1 Mark)

```
# Step6 Find the relative pose between the two cameras, i.e. find a rotation matrix R and a translation vector t such that x_2 = Rx_1 + t
import numpy as np
# Step1 Find the unit vector along the line
i = np.array([0.5, 0.4, 0.7])
j = np.array([-0.4305, 0.8907, 0.0990])
# Find the unit vector along the line
k = np.cross(i, j)
# Step2 Translate the camera coordinate system by [0, 0, 1]
t = np.array([0, 0, 1])
# Step3 Rotate the camera coordinate system about the k axis by pi/2 in a counterclockwise direction
theta = np.pi/2
# Rotation matrix
R = np.array([[np.cos(theta), -np.sin(theta), 0], [np.sin(theta), np.cos(theta), 0], [0, 0, 1]])
# Step4 Write the location of the line passing through the points [1,1,1]^T and [1,2,1]^T in the camera coordinates
# Line through the points
line = np.cross(np.array([1, 1, 1]), np.array([1, 2, 1]))
# Print the line
print("Line: ", line)
# Step5 Write the location of the line in i - j + k and i + 2j + k. What is the epipolar relationship between these two lines in the Euclidean plane? What is their intersection (hint: What is their intersection in Homogeneous Coordinates?
# Line through the points
line = np.cross(np.array([1, -1, 1]), np.array([1, 2, 1]))
# Print the line
print("Line: ", line)
# Step6 Find the relative pose between the two cameras, i.e. find a rotation matrix R and a translation vector t such that x_2 = Rx_1 + t
# Rotation matrix
R = np.array([[np.cos(theta), -np.sin(theta), 0], [np.sin(theta), np.cos(theta), 0], [0, 0, 1]])
# Translation vector
t = np.array([0, 0, 1])
# Print the translation vector
print("Translation vector: ", t)
```

```
Line:  [ 1 -1  1]
Line:  [-3  0  3]
Translation vector:  [0 0 1]
```

- **Step7** Write the intrinsic parameter matrix for the camera described by the configuration in parts **Step1** and **Step3**.

(1 Mark)

```
# Step7 Write the intrinsic parameter matrix for the camera described by the configuration in parts Step1 and Step3
import numpy as np
# Step1 Find the unit vector along the line
i = np.array([0.5, 0.4, 0.7])
j = np.array([-0.4305, 0.8907, 0.0990])
# Find the unit vector along the line
k = np.cross(i, j)
# Step2 Translate the camera coordinate system by [0, 0, 1]
t = np.array([0, 0, 1])
# Step3 Rotate the camera coordinate system about the k axis by pi/2 in a counterclockwise direction
theta = np.pi/2
# Rotation matrix
R = np.array([[np.cos(theta), -np.sin(theta), 0], [np.sin(theta), np.cos(theta), 0], [0, 0, 1]])
# Step4 Write the location of the line passing through the points [1,1,1]^T and [1,2,1]^T in the camera coordinates
# Line through the points
line = np.cross(np.array([1, 1, 1]), np.array([1, 2, 1]))
# Print the line
print("Line: ", line)
# Step5 Write the location of the line in i - j + k and i + 2j + k. What is the epipolar relationship between these two lines in the Euclidean plane? What is their intersection (hint: What is their intersection in Homogeneous Coordinates?
# Line through the points
line = np.cross(np.array([1, -1, 1]), np.array([1, 2, 1]))
# Print the line
print("Line: ", line)
# Step6 Find the relative pose between the two cameras, i.e. find a rotation matrix R and a translation vector t such that x_2 = Rx_1 + t
# Rotation matrix
R = np.array([[np.cos(theta), -np.sin(theta), 0], [np.sin(theta), np.cos(theta), 0], [0, 0, 1]])
# Translation vector
t = np.array([0, 0, 1])
# Print the translation vector
print("Translation vector: ", t)
# Step7 Write the intrinsic parameter matrix for the camera described by the configuration in parts Step1 and Step3
# Intrinsic parameter matrix
K = np.array([[1, 0, 0], [0, 1, 0], [0, 0, 1]])
# Print the intrinsic parameter matrix
print("Intrinsic parameter matrix: ", K)
```

```
Line:  [ 1 -1  1]
Line:  [-3  0  3]
Translation vector:  [0 0 1]
Intrinsic parameter matrix:  [[1 0 0]
 [0 1 0]
 [0 0 1]]
```

- Add this coordinate to these points, where $\hat{k}$ is of unit 1 from. Write all the points homogeneous by adding a fourth coordinate equal to 1.

(1 Mark)

```
# Add this coordinate to these points, where k is of unit 1 from. Write all the points homogeneous by adding a fourth coordinate equal to 1.
import numpy as np
# Step1 Find the unit vector along the line
i = np.array([0.5, 0.4, 0.7])
j = np.array([-0.4305, 0.8907, 0.0990])
# Find the unit vector along the line
k = np.cross(i, j)
# Step2 Translate the camera coordinate system by [0, 0, 1]
t = np.array([0, 0, 1])
# Step3 Rotate the camera coordinate system about the k axis by pi/2 in a counterclockwise direction
theta = np.pi/2
# Rotation matrix
R = np.array([[np.cos(theta), -np.sin(theta), 0], [np.sin(theta), np.cos(theta), 0], [0, 0, 1]])
# Step4 Write the location of the line passing through the points [1,1,1]^T and [1,2,1]^T in the camera coordinates
# Line through the points
line = np.cross(np.array([1, 1, 1]), np.array([1, 2, 1]))
# Print the line
print("Line: ", line)
# Step5 Write the location of the line in i - j + k and i + 2j + k. What is the epipolar relationship between these two lines in the Euclidean plane? What is their intersection (hint: What is their intersection in Homogeneous Coordinates?
# Line through the points
line = np.cross(np.array([1, -1, 1]), np.array([1, 2, 1]))
# Print the line
print("Line: ", line)
# Step6 Find the relative pose between the two cameras, i.e. find a rotation matrix R and a translation vector t such that x_2 = Rx_1 + t
# Rotation matrix
R = np.array([[np.cos(theta), -np.sin(theta), 0], [np.sin(theta), np.cos(theta), 0], [0, 0, 1]])
# Translation vector
t = np.array([0, 0, 1])
# Print the translation vector
print("Translation vector: ", t)
# Step7 Write the intrinsic parameter matrix for the camera described by the configuration in parts Step1 and Step3
# Intrinsic parameter matrix
K = np.array([[1, 0, 0], [0, 1, 0], [0, 0, 1]])
# Print the intrinsic parameter matrix
print("Intrinsic parameter matrix: ", K)
# Add this coordinate to these points, where k is of unit 1 from. Write all the points homogeneous by adding a fourth coordinate equal to 1.
```

```
Line:  [ 1 -1  1]
Line:  [-3  0  3]
Translation vector:  [0 0 1]
Intrinsic parameter matrix:  [[1 0 0]
 [0 1 0]
 [0 0 1]]
```

- Compute the projection of these homogeneous points using the camera matrix from part **Step1**. Convert the homogeneous projected points to 2D Euclidean points by dividing out and subsequently discarding the third coordinate of each point.

(1 Mark)

```
# Compute the projection of these homogeneous points using the camera matrix from part Step1. Convert the homogeneous projected points to 2D Euclidean points by dividing out and subsequently discarding the third coordinate of each point.
import numpy as np
# Step1 Find the unit vector along the line
i = np.array([0.5, 0.4, 0.7])
j = np.array([-0.4305, 0.8907, 0.0990])
# Find the unit vector along the line
k = np.cross(i, j)
# Step2 Translate the camera coordinate system by [0, 0, 1]
t = np.array([0, 0, 1])
# Step3 Rotate the camera coordinate system about the k axis by pi/2 in a counterclockwise direction
theta = np.pi/2
# Rotation matrix
R = np.array([[np.cos(theta), -np.sin(theta), 0], [np.sin(theta), np.cos(theta), 0], [0, 0, 1]])
# Step4 Write the location of the line passing through the points [1,1,1]^T and [1,2,1]^T in the camera coordinates
# Line through the points
line = np.cross(np.array([1, 1, 1]), np.array([1, 2, 1]))
# Print the line
print("Line: ", line)
# Step5 Write the location of the line in i - j + k and i + 2j + k. What is the epipolar relationship between these two lines in the Euclidean plane? What is their intersection (hint: What is their intersection in Homogeneous Coordinates?
# Line through the points
line = np.cross(np.array([1, -1, 1]), np.array([1, 2, 1]))
# Print the line
print("Line: ", line)
# Step6 Find the relative pose between the two cameras, i.e. find a rotation matrix R and a translation vector t such that x_2 = Rx_1 + t
# Rotation matrix
R = np.array([[np.cos(theta), -np.sin(theta), 0], [np.sin(theta), np.cos(theta), 0], [0, 0, 1]])
# Translation vector
t = np.array([0, 0, 1])
# Print the translation vector
print("Translation vector: ", t)
# Step7 Write the intrinsic parameter matrix for the camera described by the configuration in parts Step1 and Step3
# Intrinsic parameter matrix
K = np.array([[1, 0, 0], [0, 1, 0], [0, 0, 1]])
# Print the intrinsic parameter matrix
print("Intrinsic parameter matrix: ", K)
# Add this coordinate to these points, where k is of unit 1 from. Write all the points homogeneous by adding a fourth coordinate equal to 1.
# Compute the projection of these homogeneous points using the camera matrix from part Step1. Convert the homogeneous projected points to 2D Euclidean points by dividing out and subsequently discarding the third coordinate of each point.
```

```
Line:  [ 1 -1  1]
Line:  [-3  0  3]
Translation vector:  [0 0 1]
Intrinsic parameter matrix:  [[1 0 0]
 [0 1 0]
 [0 0 1]]
```

```
----> 2 P = np.matmul(project_matrix,Fresult)
      3 P1 = np.matmul(K,Projection2)
      4 P2 = np.matmul(P1,P)

NameError: name 'Fresult' is not defined
```

4. Plot the projected 2D points, again ensure that the axes of your plot are equal. What is the shape of the image of a circle?

[2 Points]

In [236…

```
plt.scatter(x_r, y_r)

plt.show()
# ellipse
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Input In [236], in <cell line: 1>()
----> 1 plt.scatter(x_r, y_r)
      3 plt.show()

NameError: name 'x_r' is not defined
```